

HopIn: A Scalable MERN-Based Carpooling Platform for Smart Urban Mobility

Dissertation submitted to the

University College of Engineering (A), Osmania University.

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

DEVULAPALLY KESHAV SAI RAO (100521733012)

KAMLEKAR SAI SIDDHARTH (100521733024)

RAMCHANDAR RAO SWARGAM (100521733064)

Under the Guidance of

Prof. B. Sujatha

Dept. of CSE, UCEOU.



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**UNIVERSITY COLLEGE OF ENGINEERING (A),
OSMANIA UNIVERSITY HYDERABAD, TS-500007**

MAY - 2025



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING,
OSMANIA UNIVERSITY**

CERTIFICATE

This is to certify the bonafide work of **Devulapally Keshav Sai Rao, Kamlekar Sai Siddharth and Ramchandrar Rao Swargam** bearing roll numbers **100521733012, 100521733024, 100521733064**, for their major project, in partial fulfilment of Bachelor of Engineering Degree, offered by Department of Computer Science and Engineering, University College of Engineering, Osmania University.

Project Guide

Prof. B. Sujatha
Dept. of CSE, UCEOU

Head of the Department

Prof. P.V Sudha
Dept. of CSE, UCEOU

STUDENT DECLARATION

We declare that the work reported in the project report entitled “HopIn: A Scalable MERN-Based Carpooling Platform for Smart Urban Mobility” submitted by Devulapally Keshav Sai Rao, Kamlekar Sai Siddharth and Ramchandrarao Swargam is a record of the work done by us in the DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING, UNIVERSITY COLLEGE OF ENGINEERING . No part of the report is copied from books/journals/internet and wherever referred, the same has been duly acknowledged in the text . The reported data is based on the work done entirely by us and not copied from any other source or submitted to any other Institute or University for the award of a degree or diploma.

SIGNATURES

Devulapally Keshav Sai Rao :

Kamlekar Sai Siddharth :

Ramchandrarao Swargam :

ACKNOWLEDGMENT

It is our privilege and pleasure to express our profound sense of respect, gratitude, and indebtedness to our guide **Prof. B. Sujatha**, Department of Computer Science and Engineering, UCEOU, for his inspiration, guidance, cogent discussion, constructive criticisms, and encouragement throughout this dissertation work.

We also thank **Prof. P. V. Sudha**, Head Department of Computer Science and Engineering for her support from the Department and for making all the resources available to us students. We also extend my thanks to the entire faculty of the Department of Computer Science and Engineering, University College of Engineering, Osmania University who encouraged us throughout the course of our Bachelor degree and allowed us to use the many resources present in the department. Our sincere thanks to our parents and friends for their valuable suggestions, morals, strength, and support for the completion of our project

ABSTRACT

In today's fast-paced and dynamic healthcare environment, time is of the essence, and every moment wasted can have significant repercussions for both patients and medical professionals. By automating the entire process of obtaining an X-ray, analyzing it for fractures, and determining the appropriate course of action, our system aims to streamline patient care, minimize unnecessary visits to the doctor, and maximize the efficient allocation of medical resources.

Through the implementation of advanced artificial intelligence technologies, our project not only addresses the immediate need for improved fracture detection but also sets the stage for a transformative shift in healthcare delivery. By leveraging the capabilities of AI to navigate the complexities of fracture diagnosis with speed and precision, we empower healthcare providers to deliver timely and effective care while optimizing their workflows and resources.

Furthermore, by reducing the burden of manual tasks and administrative duties associated with fracture diagnosis, our system frees up valuable time for medical professionals to focus on more critical patient cases, ultimately enhancing overall productivity and patient outcomes.

In essence, our project represents a bold step towards a future where AI-driven automation plays a central role in revolutionizing healthcare delivery, ensuring that patients receive the timely and accurate care they need while maximizing efficiency and resource utilization within the healthcare system.

TABLE OF CONTENTS

S.NO	CONTENT	PAGE NO.
1.	Chapter 1: Introduction 1.1. Introduction 1.2. Objective 1.3. Need for Carpooling 1.4. MERN Stack Overview 1.5. Advantages of MERN in HopIn	1 1 1 2 2 3
2.	Chapter 2 : Literature Survey 2.1 Introduction to Literature Survey 2.2 Existing Research on Carpooling Systems 2.3 Ride Sharing Platforms Overview 2.4 Limitations of Existing Systems 2.5 Research Gaps and Motivation 2.6 Shared Mobility and India's Generation Z : Environmental Consciousness, Risks and Attitudes 2.7 Summary	5 5 6 7 7 8 12
3.	Chapter 3 : Existing Systems 3.1 Introduction 3.2 Analysis of Current Platforms 3.3 Feature Comparison Matrix 3.4 Observations and Insights 3.5 Conclusion	13 13 15 15 16
4.	Chapter 4: Proposed Systems	

	4.1 System Overview	17
	4.2 Architecture Diagram	17
	4.3 Key Functional Modules	18
	4.4 Technologies used	19
	4.5 Database Schema Design	20
	4.6 Security Features	21
	4.7 Real time Communication(Optional Enhancement)	21
	4.8 System Advantages	21
	4.9 User Experience Flow	21
	4.10 Summary	22
5.	Chapter 5: Implementation	
	5.1 Front End Development with React.js	23
	5.2 Backend Development with Node.js and Express.js	24
	5.3 MongoDB and Mongoose Integration	26
	5.4 Sample Screens and User Interface	27
	5.5 Deployment and Hosting	27
	5.6 Challenges and Solutions	28
	5.7 Conclusion	28
6.	Chapter 6: Results	
	6.1 Functional Validation	29
7.	Chapter 7: Conclusion and Future Scope	
	7.1 Conclusion	35
	7.2 Future Scope	36
	7.3 Final Thoughts	37

8.	References	38
9.	Appendix	40

LIST OF FIGURES

Fig. No.	Description	Page No.
Fig. 1	Conceptual Framework	11
Fig. 2	Generation Z's awareness of different shared mobility platforms in India	11
Fig. 3	Architecture of System based on MERN Stack	18
Fig. 4	Sign Up form - HopIn	30
Fig. 5	Sign In form - HopIn	30
Fig. 6	Home Page - HopIn	31
Fig. 7	Create Ride Page - HopIn	31
Fig. 8	Find Ride Page - HopIn	32
Fig. 9	Profile Page - HopIn	33
Fig. 10	Load on Database	34
Fig. 11	Architecture of MERN Stack	34

CHAPTER I

1.INTRODUCTION

1.1 INTRODUCTION

Urban mobility is undergoing a transformation driven by the convergence of digital platforms, the sharing economy, and environmental imperatives. With population growth and rapid urbanization, traffic congestion and pollution have become critical issues in cities across the globe. Single-occupancy vehicles dominate the roads, wasting resources and adding to the carbon footprint. There is a pressing need for intelligent, accessible, and scalable commuting alternatives that optimize road usage and enhance user convenience.

Carpooling, also known as ride-sharing, is a promising solution to these problems. It allows individuals with similar travel routes and timings to share a vehicle, splitting costs and reducing traffic density. However, existing carpooling systems are often fragmented, difficult to use, or fail to provide reliable real-time services. This creates an opportunity for modern, app-based systems that use cutting-edge technology to improve the user experience and adoption rate.

Our project, **HopIn**, is developed with these motivations in mind. It is a full-stack web application designed to connect drivers and riders seamlessly, enabling them to schedule, match, and manage shared rides through an intuitive platform. By leveraging the MERN (MongoDB, Express.js, React.js, Node.js) stack, HopIn delivers a robust and scalable system with a smooth and responsive user interface, secure backend APIs, and real-time ride coordination.

HopIn aims to address modern commuting problems with a system that is:

- **Efficient** in terms of reducing travel time and traffic congestion.
- **Cost-effective** for users through shared travel expenses.
- **Eco-friendly** by reducing vehicle emissions.
- **Community-driven** by connecting people with similar routes.

The platform is designed to be mobile-responsive, ensuring accessibility across devices. It encourages the adoption of ride-sharing as a lifestyle change rather than just a service, promoting long-term benefits for users and cities.

1.2 OBJECTIVE

The primary objective of HopIn is to design and implement a dynamic, user-friendly web-based platform that enables:

- **Real-time ride matching** between drivers and riders with similar routes and schedules.
- **Secure and intuitive user registration and login** mechanisms with validation and encryption.
- **Trip creation and search features** to allow drivers to post rides and riders to find them using various filters.
- **Efficient ride booking and cancellation** functionality with real-time seat availability updates.
- **An interactive admin dashboard** for managing users, monitoring trips, and generating analytics.

Other core goals include:

- **Data security** through secure APIs and token-based authentication.
- **Scalability** to accommodate large user bases.
- **Performance optimization** for faster load times and responsiveness.

1.3 NEED FOR CARPOOLING

India's urban roads are overburdened. According to a 2023 traffic congestion index, major Indian cities like Bengaluru, Mumbai, and Delhi are among the world's most congested cities. In addition to long commute times, air quality levels have deteriorated due to excessive fuel consumption.

Carpooling offers several benefits:

- **Reduction in the number of vehicles** on roads, easing congestion.
- **Lower fuel consumption and emissions**, promoting greener travel.
- **Cost-sharing benefits** for riders and drivers.
- **Enhanced social interaction** through shared rides.
- **Optimal resource usage**, especially during peak hours.

Despite these benefits, the adoption of carpooling in India is still low due to factors like trust issues, lack of awareness, difficulty in finding suitable co-riders, and absence of structured digital platforms. HopIn addresses these challenges by providing a secure, verified, and user-friendly platform, increasing trust and ease of use among commuters.

1.4 MERN STACK OVERVIEW

The MERN stack is a collection of powerful technologies that help developers build full-stack applications using JavaScript across all layers. The stack includes:

- **MongoDB:** A NoSQL database designed for scalability and flexibility. It stores data in BSON (binary JSON), making it ideal for handling semi-structured data like user profiles, ride details, and booking histories.
- **Express.js:** A web application framework for Node.js. It simplifies the creation of backend services and APIs through built-in middleware and robust routing mechanisms.
- **React.js:** A JavaScript library for building interactive and reusable UI components. It uses a virtual DOM to enhance performance and provides hooks and context APIs for advanced state management.
- **Node.js:** A JavaScript runtime built on Chrome's V8 engine. It enables server-side scripting and event-driven architecture, perfect for asynchronous processing and real-time features.

MERN allows for:

- **End-to-end JavaScript development**
- **Rapid prototyping and development cycles**
- **Easier debugging and testing** due to unified language stack
- **Support for microservices** and scalable backend design

MERN is well-suited for applications like HopIn that require real-time communication, dynamic content updates, and a modular architecture.

1.5 ADVANTAGES OF MERN IN HOPIN

The MERN stack was selected based on its performance, ease of integration, and community support. Key benefits in the context of HopIn include:

- **Modular and Component-Based Architecture:** With React on the frontend, the application can be split into reusable modules (Login, RideSearch, TripCard, Dashboard, etc.), improving maintainability.
- **Rapid Development:** Unified language and vast open-source support reduce development time.
- **Real-Time Functionality:** Node.js supports real-time features such as seat updates, notifications, and user tracking.
- **Scalability:** MongoDB's horizontal scaling and Node's asynchronous architecture support thousands of concurrent users.
- **Security:** Express.js allows for integration of security features such as JWT (JSON Web Tokens), OAuth, HTTPS, and data validation.
- **Cross-Platform Deployment:** MERN applications are easily deployable on various platforms including AWS, Heroku, and Vercel.

In HopIn, MongoDB is used to store:

- User profiles (including authentication credentials)
- Trip records (with origin, destination, time, etc.)
- Booking history and ride status

Express.js and Node.js manage API requests such as:

- Posting and retrieving ride listings
- Processing ride bookings
- Authenticating user sessions

React.js enhances user engagement with:

- Dynamic filtering and sorting of trips
- Responsive design for mobile and desktop views
- Error handling and loading indicators for improved UX

CHAPTER II

2.LITERATURE SURVEY

2.1 INTRODUCTION TO LITERATURE SURVEY

A literature survey is essential for understanding the current state of research and technology in the domain of carpooling systems and ride-sharing applications. It provides insight into the challenges, solutions, and technological evolutions that have shaped current systems. This chapter explores academic papers, industry implementations, and user-centric studies that have influenced the architecture and design philosophy of HopIn.

By reviewing existing literature, we aim to:

- Identify gaps in current systems
- Analyze various carpooling models and platforms
- Assess user expectations and behavioral patterns
- Evaluate the success of different technological approaches

2.2 EXISTING RESEARCH ON CARPOOLING SYSTEMS

Numerous research initiatives have evaluated the effectiveness and challenges of carpooling systems. Key themes across literature include:

- **Trust and Safety Mechanisms:** Studies consistently highlight the need for safety features such as driver verification, ride history tracking, and user reviews. These measures are vital in increasing user confidence and adoption.
- **Dynamic Matching Algorithms:** Algorithms like Dijkstra, A*, and Machine Learning classifiers are utilized to match riders and drivers efficiently based on proximity, route similarity, and time constraints.
- **Behavioral and Social Factors:** Research by *Bhat and Sardesai (2006)* indicates that ride-sharing success hinges significantly on social compatibility, convenience, and financial incentives.

Case Studies:

- *Jain and Mishra (2020)* emphasized that real-time traffic integration significantly enhances route accuracy and user satisfaction.
- *Kumar et al. (2019)* simulated traffic models showing a 30–40% reduction in vehicle density with successful ride-sharing in tier-1 Indian cities.
- *Verma and Manoj (2017)* found that app-based systems were 60% more likely to be adopted than traditional web-based systems.

2.3 RIDE-SHARING PLATFORMS OVERVIEW

The rise of commercial carpooling solutions provides valuable insights into what works and what doesn't. Key platforms include:

- **UberPool:** Offers on-demand ride-sharing within urban areas. It uses advanced route matching algorithms and predictive analytics but lacks the flexibility for community-level operations.
- **BlaBlaCar:** Focuses on long-distance inter-city travel. Features such as user profiles, identity verification, and reviews build trust but are not optimized for short, daily commutes.
- **QuickRide:** Primarily targets working professionals and integrates with corporate campuses. While it ensures security through official email verifications, it has limited flexibility outside the workplace.

Feature	UberPool	BlaBlaCar	QuickRide	HopIn
Target Market	General public	Intercity	Corporate	Urban commuters
Trust Mechanisms	Ratings	ID Verified	Enterprise-only	Ratings + Admin
Route Flexibility	Medium	Low	Low	High
Platform Type	Mobile	Web + Mobile	Mobile	Web + Mobile
Real-time Matching	Yes	No	Limited	Yes
Admin Oversight	No	No	Partial	Full

2.4 LIMITATIONS OF EXISTING SYSTEMS

While popular platforms solve key mobility issues, several limitations persist:

- **Data Privacy and Transparency:** Lack of clarity on data storage, usage, and third-party sharing raises concerns among users.
- **Route Customization:** Fixed pickup/drop locations and schedules may not suit users with flexible or varying routines.
- **Inconsistent Service Quality:** User experiences can vary drastically due to unverified drivers or riders, and lack of moderation.
- **Infrastructure Dependency:** Some apps rely heavily on GPS or corporate networks, limiting accessibility in under-connected areas.

Furthermore, high dependency on centralized architectures can create bottlenecks and increase maintenance complexity.

2.5 RESEARCH GAPS AND MOTIVATION

A thorough review of current literature and market offerings reveals key gaps that HopIn seeks to fill:

1. **Lack of Modular, Open Source Implementations:** Most systems are proprietary and difficult to replicate for research or academic use.
2. **Limited Admin Involvement:** Platforms lack dedicated admin dashboards for controlling abuse, managing disputes, and monitoring ride trends.
3. **Cultural Context:** Few platforms are tailored for Indian urban commuters who value personalization, flexibility, and safety.
4. **Real-Time Communication:** Features like live chat, in-ride updates, and seat availability are often rudimentary or missing.

HopIn is designed to overcome these issues by:

- Employing a fully open-source MERN stack framework.
- Integrating admin dashboards with real-time user control.
- Building flexible ride-matching based on micro-location and preferred filters.
- Emphasizing transparency, scalability, and personalized experiences.

2.6 SHARED MOBILITY AND INDIA'S GENERATION Z: ENVIRONMENTAL CONSCIOUSNESS, RISKS, AND ATTITUDES

Overview:

This study examines the adoption and perception of shared mobility platforms among Generation Z in India. With growing concerns over environmental sustainability, urban congestion, and economic inefficiencies, shared mobility platforms (e.g., Uber, Ola, Rapido) have emerged as viable alternatives to car ownership. The paper focuses on Generation Z's (born ~1997–2012) attitudes, motivations, and perceived risks regarding these platforms.

Significance:

Generation Z is digitally native, socially conscious, and environmentally aware—traits that position them as ideal candidates for shared economy participation. Their preferences for convenience, low cost, and sustainability, along with reduced interest in car ownership, make this demographic key to understanding and advancing the shared mobility ecosystem in India.

Research Aim:

To explore how environmental consciousness, social and economic benefits, and perceived risks influence Gen Z's intention to use shared mobility platforms, and how these factors collectively shape their attitudes and behaviors.

Methodology and Demographics

Methodology:

A quantitative approach was adopted using Partial Least Squares Structural Equation Modeling (PLS-SEM). A structured questionnaire was administered to 318 Generation Z individuals across Indian colleges, with respondents having at least five travel sessions per month on mobility platforms.

Demographics Summary:

- **Gender:** 58% Female, 41% Male
- **Location:** 83% Urban residents
- **Education:** 63% undergraduates
- **Monthly expenditure:** 57% spend below ₹10,000
- **Travel spending:** 61% spend under ₹2,500/month

Platform Awareness:

- **Top Platforms:** Uber (94%), Ola (93%), Rapido (84%)
- **Less Known:** BluSmart Mobility (3.8%), BlaBlaCar (8.2%)

Travel Preferences:

- **Short distances:** 57% prefer walking/cycling
- **Long distances:** 33% prefer ride-hailing (e.g., Uber/Ola)

Sources of Awareness:

- Social media (48%) and YouTube ads (28%) dominate, reflecting Gen Z's digital habits.

Key Findings and Analysis

Key Constructs Measured:

1. Environmental Consciousness:

Gen Z values pollution reduction, sustainability, and resource conservation. High agreement indicates a strong moral inclination toward eco-friendly transportation.

2. Social Aspects:

Security, reduced waiting time, and social good (e.g., access for those without vehicles) are important. Booking convenience and safety features increase platform trust.

3. Economic Benefits:

Cost savings (fuel, parking, maintenance), flexible payment options, and vehicle variety are appreciated. Shared mobility is seen as a way to avoid ownership burdens.

4. Perceived Risks:

Concerns include vehicle availability, territorial coverage, unfamiliar controls, and future reliability with increasing automation.

Notable Insight:

Despite risks, Gen Z remains inclined toward shared mobility due to the strong influence of environmental and economic motivations.

Implications, Conclusions, and Recommendations

Theoretical Implications:

- Introduces a validated model to predict Gen Z's adoption of shared mobility.
- Highlights the interplay between environmental values, economic rationale, and social considerations.

Practical Implications:

- **Businesses:**
 - Reduce perceived risks (expand coverage, improve availability).
 - Market environmental benefits and economic value.
 - Emphasize safety and social impact in branding.
- **Policy Makers:**
 - Provide incentives (e.g., tax rebates, infrastructure support).
 - Promote carpooling, e-vehicles, and congestion reduction strategies.

Conclusion:

Generation Z demonstrates a strong potential to drive sustainable transport adoption through shared mobility. Their environmentally conscious, cost-aware, and tech-savvy nature aligns with the values shared platforms offer. For shared mobility to thrive, addressing operational risks and enhancing digital engagement strategies are crucial.

Limitations & Future Research:

- **Sampling Bias:** Convenience sampling limits generalizability.
- **Scope:** Urban-centric focus; rural/suburban insights needed.
- **Cross-national Comparisons:** Studies in other countries could reveal broader trends.

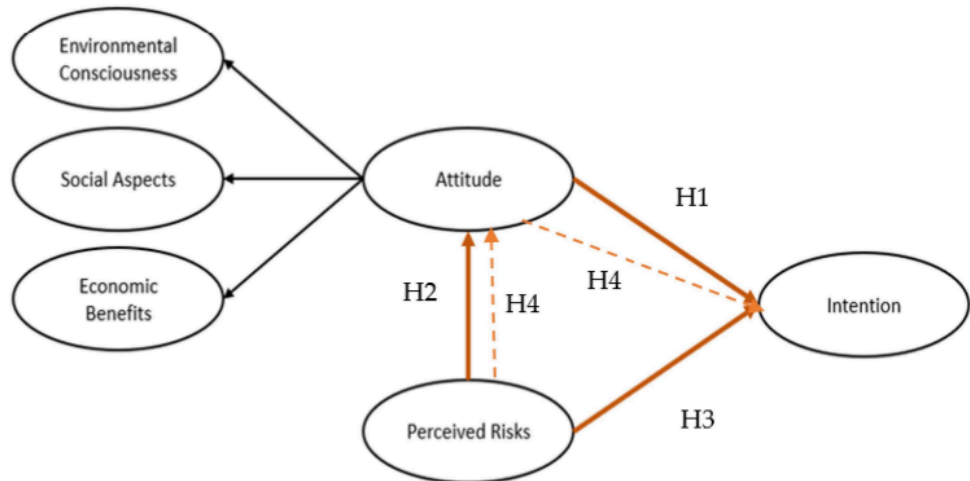


Figure 1. Conceptual framework (authors' own elaboration).

Table 1. Demographic profile of the respondents.

		Count	Column N %
Gender	Male	131	41.2%
	Female	184	57.9%
	Other	3	0.9%
Educational qualification	Class X/XII	84	26.4%
	Undergraduate (UG)	199	62.6%
	Postgraduate (PG)	35	11.0%
Area of stay	Rural	18	5.7%
	Suburb	35	11.0%
	Urban	265	83.3%
Monthly expenditure level (in ₹)	Below 10,000	180	56.6%
	10,000–20,000	93	29.2%
	20,000–30,000	27	8.5%
	Above 30,000	18	5.7%
Monthly travel expenditure level (in ₹)	Below 2500	195	61.3%
	2500–5000	86	27.0%
	5000–7500	17	5.3%
	7500–10,000	10	3.1%
	Above 10,000	10	3.1%

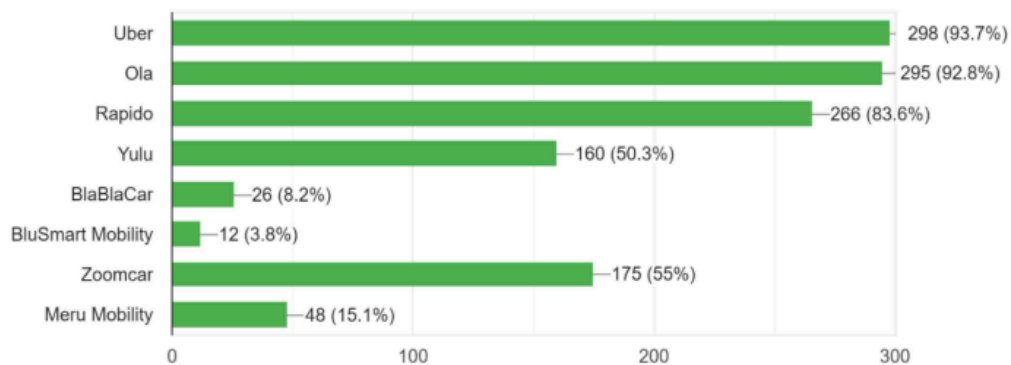


Figure 2. Generation Z's awareness of different shared mobility platforms in India (authors' own elaboration).

2.7 SUMMARY

The literature survey underscores the evolution, challenges, and gaps in current ride-sharing systems. It has revealed valuable insights into what commuters expect from such platforms and what current solutions lack. HopIn positions itself as a modern, flexible, and secure alternative tailored for daily urban commuting. The next chapter explores existing solutions in greater detail and highlights the innovative contributions of our proposed system.

CHAPTER III

3.EXISTING METHODS

3.1 INTRODUCTION

Before conceptualizing and designing HopIn, it is essential to analyze and understand the functioning and limitations of existing carpooling systems. This chapter explores the architecture, user experience, features, and constraints of popular ride-sharing applications. It aims to provide a comparative study to position HopIn as a more robust, flexible, and user-oriented platform.

3.2 ANALYSIS OF CURRENT PLATFORMS

Several ride-sharing platforms currently serve different segments of the commuting population. The most widely used ones include UberPool, BlaBlaCar, QuickRide, and RideAmigos. Below is an elaborated review of each platform:

3.2.1 UberPool

UberPool offers an option for riders to share rides with others heading in the same direction. It reduces fare costs and is integrated into the Uber mobile application.

- Strengths:
 - Uses real-time traffic data for efficient routing
 - Predictive demand algorithms improve matching
 - Wide market reach and brand recognition
- Limitations:
 - No admin panel for ride moderation
 - Lack of customization for rider preferences
 - Not available in all cities or during all times

3.2.2 BlaBlaCar

BlaBlaCar targets inter-city travelers for long-distance ride-sharing.

- Strengths:
 - Comprehensive user verification
 - Flexible pricing system
 - Community-driven reviews enhance trust
- Limitations:
 - Inefficient for daily short commutes
 - Low user engagement in some regions
 - Interface not optimized for urban transport

3.2.3 QuickRide

QuickRide is designed for office commuters, especially in metropolitan Indian cities.

- Strengths:
 - Strong corporate network integration
 - In-app wallet for payment handling
 - Employee verification through email IDs
- Limitations:
 - Not suitable for general public use
 - Restricted to working hours and routes
 - User interface is outdated and less intuitive

3.2.4 RideAmigos

RideAmigos focuses on institutional ride-sharing programs including universities and government organizations.

- Strengths:
 - Advanced trip-planning and analytics
 - Suitable for event-specific ride coordination
- Limitations:
 - Complex interface for first-time users
 - Primarily used by institutions, not individuals

3.3 FEATURE COMPARISON MATRIX

To better understand where HopIn improves upon these platforms, the following feature matrix is presented:

Feature	UberPool	BlaBlaCar	QuickRide	RideAmigos	HopIn
Real-time Matching	Yes	No	Limited	Partial	Yes
Long/Short Distance	Short	Long	Short	Event-based	Both
Admin Dashboard	No	No	Limited	Yes	Yes
Trip Customization	Low	Medium	Low	Medium	High
User Verification	Medium	High	High	High	High
Platform Type	Mobile	Web/Mobile	Mobile	Web	Web/Mobile
Payment Integration	Yes	Yes	Yes	No	Optional

3.4 OBSERVATIONS AND INSIGHTS

The comparison and detailed analysis reveal the following insights:

- There is a gap in platforms that support both urban and inter-city ride-sharing efficiently.
- Most platforms lack a centralized admin system for moderation and data analytics.
- Customizability and flexibility are often under-prioritized in favor of rigid templates.
- User interfaces vary in complexity, and user onboarding experiences are not always intuitive.
- Real-time capabilities, such as seat tracking and live notifications, are often inconsistent or unavailable.

3.5 CONCLUSION

While current systems address specific needs, none provide a holistic, customizable, and scalable solution tailored for daily urban commuters in a community-driven context. HopIn is uniquely positioned to fill this gap. It combines advanced technologies, user-centric design, and real-time functionality with administrative oversight, which will be further detailed in the next chapter discussing the proposed system architecture.

CHAPTER IV

4.PROPOSED METHOD

4.1 SYSTEM OVERVIEW

The proposed system, HopIn, is a real-time carpooling platform designed to connect riders and drivers effectively. HopIn serves as a bridge between passengers who seek cost-effective travel and drivers willing to share vacant seats in their vehicles. The objective is to minimize the number of single-occupancy vehicles on the road, reduce traffic congestion, and provide an environmentally sustainable commuting option.

HopIn is developed using the MERN stack:

- MongoDB for the database
- Express.js for the web framework
- React.js for the user interface
- Node.js for the runtime and backend logic

The system follows a modular, scalable, and service-oriented architecture, making it highly maintainable and suitable for cloud deployment.

4.2 ARCHITECTURE DIAGRAM

The architecture of HopIn follows the classic 3-tier model:

1. Presentation Layer: The React frontend interfaces with users.
2. Application Layer: Node.js + Express manages business logic and routes.
3. Data Layer: MongoDB handles all data storage in collections.

This modularity allows the system to scale horizontally by separating frontend, backend, and database into different services (e.g., using Docker containers or cloud microservices).

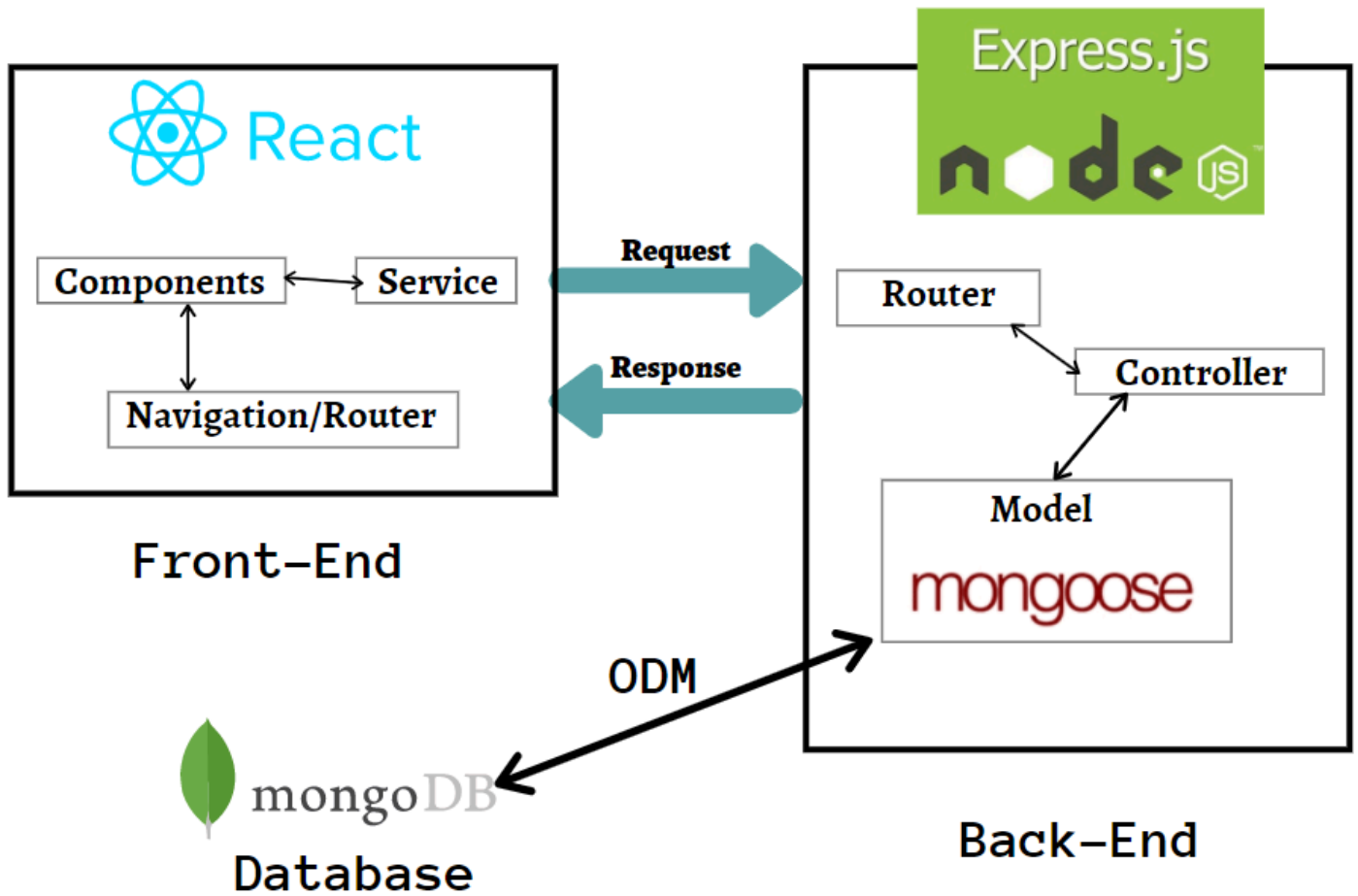


Fig 3: Conceptual Framework

4.3 KEY FUNCTIONAL MODULES

4.3.1 Authentication Module

- Allows users to register, login, and reset passwords
- Uses JWT (JSON Web Tokens) for stateless authentication
- Includes role-based access control to separate riders and drivers

4.3.2 Ride Creation and Search Module

- Drivers can post new rides with:
 - Pickup and drop locations
 - Available seats

- Departure time and fare
- Riders can search for available rides using:
 - Filters such as origin, destination, time range, and seat availability

4.3.3 Booking and Matching Engine

- Riders can request to book a seat in a ride
- Drivers receive notifications of booking requests and can accept or reject
- A real-time matching system ensures only compatible matches are shown

4.3.4 User Dashboard

- Shows ride history, upcoming bookings, and booking status
- Separate dashboards for drivers and riders
- Allows cancellation of trips/bookings with policy enforcement

4.3.5 Admin Panel

- Monitors user activity
- Manages complaints and abuse reports
- Performs analytics like most active users/routes, usage trends

4.4 TECHNOLOGIES USED

Layer	Technology	Purpose
Frontend	React.js	UI development
Backend	Node.js, Express	Server-side logic
Database	MongoDB	Data storage (users, rides, etc.)

Authentication	JWT, bcrypt	Secure login system
Styling	TailwindCSS	Responsive design
Deployment	Netlify, Render	Cloud deployment
APIs	Google Maps API	Address autocomplete, routes

4.5 DATABASE SCHEMA DESIGN

The system uses a NoSQL document schema, offering flexibility as new rides or user attributes are introduced.

Key Collections:

1. Users

- userID, name, email, phone, passwordHash, userType, createdAt, vehicle, rating

2. Rides

- rideID, driverID, origin, destination, availableSeats, departureTime, fare, status, originPlaceID, destinationPlaceID

3. Bookings

- bookingID, rideID, status, timestamp, userID, seatsBooked, totalCost

4. Review

- reviewID, fromUser, toUser, rideID, rating, comments

Schema Flexibility

MongoDB allows nested documents (e.g., ride preferences) and supports geospatial indexing, useful for location-based queries. Also it creates a new id for each entity created, helping in

avoiding the problem of maintaining the primary key for each table, i.e the uniqueId generated itself acts as the primary key.

4.6 SECURITY FEATURES

- Password hashing with bcrypt
- JWT-based route protection for session security
- Input validation using express-validator
- Rate limiting to prevent brute-force attacks
- HTTPS enforced on production deployments

4.7 REAL-TIME COMMUNICATION (OPTIONAL ENHANCEMENT)

In future versions, integration with Socket.IO is planned for:

- Real-time ride status updates
- Notifications for booking approvals or cancellations
- Live chat between rider and driver

4.8 SYSTEM ADVANTAGES

- Scalable across locations and user bases
- Responsive design ensures compatibility on mobile and desktop
- Secure and encrypted transactions with user verification
- Environmentally conscious by promoting ride sharing
- Cost-efficient by splitting travel expenses

4.9 USER EXPERIENCE FLOW

1. Rider:

- Logs in → Searches for rides → Books seat → Receives confirmation

2. Driver:

- Logs in → Creates a ride → Reviews bookings → Starts trip

3. Admin:

- Monitors activity → Handles reports → Generates analytics

4.10 SUMMARY

This chapter provides a comprehensive blueprint of the HopIn platform, highlighting its modular design, technical implementation, and user-driven features. It illustrates how the MERN stack can be leveraged to develop a real-time, secure, and scalable solution for daily commuting needs.

CHAPTER V

5.IMPLEMENTATION

5.1 FRONT END DEVELOPMENT WITH REACT.JS

HopIn's frontend is built using **React.js**, a modern JavaScript library designed for building fast, dynamic, and scalable user interfaces. React's **component-based architecture** allows the team to modularize UI elements and reuse them across different parts of the application, resulting in maintainable and efficient code.

5.1.1 Major Components and Their Roles

Component	Description
authentication.jsx	Authenticates user credentials and handles secure login and registration.
bookingForm.jsx	Allows users (riders) to book a ride and submit relevant details.
bookRide.jsx	Displays ride details and booking options to the user.
createRide.jsx	Enables drivers to post new rides with time, fare, seats, and location info.
SearchRideForm.jsx	Lets users filter and search available rides based on route and time criteria.
OfferRideForm.jsx	A reusable form interface for creating or updating ride offers.
MapDisplay.jsx	Shows pickup/drop locations using Google Maps integration (if implemented).
Navbar.jsx	Navigation bar for the app, with conditional rendering based on login role.
HeroSection.jsx	The main landing page section with introduction and call to action.

Profile.jsx	Displays the user's personal information, account type, and ride history.
HomePage.jsx	Entry point after login, showing summarized dashboard content.
Footer.jsx	Contains footer links and contact/social info.

5.1.2 Key Technologies and Libraries

- **React Router DOM:** Implements page navigation using single-page application behavior.
- **Axios:** For HTTP requests to the backend.
- **Context API + useReducer:** Global state management for user authentication and session tracking.
- **Tailwind CSS:** Enables rapid styling with a mobile-first design approach.

5.1.3 Features in UI

- Mobile-first responsive layouts.
- Dark/light theme toggle.
- Toast notifications for success/error messages.
- Role-based views (Rider, Driver, Admin).
- Booking status shown in real-time: pending, accepted, rejected.

5.2 BACKEND DEVELOPMENT WITH NODE.JS AND EXPRESS.JS

The server-side logic of HopIn is implemented using **Node.js**, a non-blocking, event-driven environment, and **Express.js**, a minimalist web framework.

5.2.1 Project Directory Structure

```
backend/  
├── controllers/  
├── routes/  
├── models/  
├── middleware/  
├── utils/  
└── server.js
```

5.2.2 Core API Routes

Endpoint	Method	Description
----------	--------	-------------

/api/user/login	POST	Authenticates a user (rider/driver) and returns a JWT token.
-----------------	------	--

/api/user/register	POST	Registers a new user (with role: rider or driver).
--------------------	------	--

/api/rides	GET	Fetches a list of all available ride offers.
------------	-----	--

/api/rides/create	POST	Allows a driver to create and publish a new ride offer.
-------------------	------	---

/api/rides/:id	GET	Retrieves detailed information about a specific ride.
----------------	-----	---

/api/bookings	POST	Allows a rider to request booking for a ride.
---------------	------	---

/api/bookings/:id/status	PUT	Enables a driver to accept or reject a booking request.
--------------------------	-----	---

/api/user/:id GET Retrieves a user's profile using their unique user ID.

User Authentication (Login/Sign up)

```
const User = require('../models/users.model');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
require('dotenv').config();

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    console.log("Login request body:", req.body);

    // Check if the email and password were provided
    if (!email || !password) {
      return res.status(400).json({
        error: "Please provide both email and password."
      });
    }

    // Find the user by email
    const user = await User.findOne({ email });
    if (!user) {
      console.log("User not found for email:", email);
      return res.status(401).json({ error: "Invalid email or password." });
    }

    // Compare the provided password with the stored hashed password
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      console.log("Password mismatch for user:", email);
```

```

    return res.status(401).json({ error: "Invalid email or password." });
  }
  console.log("JWT Secret:", process.env.JWT_SECRET);

  // Generate JWT token
  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '7d' }
  );

  // Send response with user details and token
  res.status(200).json({
    message: "User logged in successfully",
    user: {
      id: user._id,
      name: user.name,
      email: user.email,
      role: user.role,
      vehicle: user.vehicle // Include vehicle info if applicable
    },
    token
  });
} catch (error) {
  console.error("Login error:", error);
  res.status(500).json({ error: error.message });
}
};

exports.registerUser = async (req, res) => {
  try {

```

```

const { name, email, password, phone, role } = req.body;

// Check for required fields
if (!name || !email || !password || !phone) {
  return res.status(400).json({
    error: "Please provide all required fields: name, email, password, and phone."
  });
}

const hashedPassword = await bcrypt.hash(password, 10); // 10 is the salt rounds

const newUser = new User({
  name,
  email,
  password: hashedPassword, // Use hashedPassword here
  phone,
  role,
});

await newUser.save();

// Send response after saving the user
res.status(201).json({
  message: 'User registered successfully',
  user: {
    id: newUser._id,
    name: newUser.name,
    email: newUser.email,
    role: newUser.role,
  }
});

```

```

    } catch (error) {
      console.error("Registration error:", error);
      res.status(500).json({ error: error.message });
    }
  };

```

Ride's Booking and Matching

```

const Ride = require('../models/ride.model');
const Booking = require('../models/booking.model');
const jwt = require('jsonwebtoken');
const User = require('../models/users.model');

const { Client } = require('@googlemaps/google-maps-services-js');
const polyline = require('@mapbox/polyline');
const client = new Client({});

exports.createRide = async (req, res) => {
  const { origin, originPlaceId, destination, destinationPlaceId, departureTime, availableSeats,
    pricePerSeat, vehicle } = req.body;

  // Validate required fields
  if (!origin || !destination || !departureTime || !availableSeats || !pricePerSeat || !vehicle ||
    !originPlaceId || !destinationPlaceId) {
    return res.status(400).json({ message: 'All fields are required' });
  }

  const newRide = new Ride({
    driver: req.user.id, // Use the user ID from the decoded token
    origin,
    originPlaceId,
    destination,
    destinationPlaceId,
    departureTime,

```

```

    availableSeats,
    pricePerSeat,
    vehicle
  });

  try {
    await newRide.save();
    res.status(201).json(newRide);
  } catch (error) {
    console.error('Error creating ride:', error);
    res.status(500).json({ message: 'Error creating ride', error: error.message });
  }
};

const GOOGLE_API_KEY = "AIzaSyDBUN4PfqDgLunQEbsZVsxslaraGmgtnqg";
function getDistanceFromLatLonInKm(lat1, lon1, lat2, lon2) {
  const R = 6371; // Earth radius in km
  const dLat = deg2rad(lat2 - lat1);
  const dLon = deg2rad(lon2 - lon1);
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(deg2rad(lat1)) * Math.cos(deg2rad(lat2)) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}

function deg2rad(deg) {
  return deg * (Math.PI / 180);
}

exports.matchRides = async (req, res) => {

```

```

const { origin, destination } = req.body;

if (!origin?.place_id || !destination?.place_id) {
  return res.status(400).json({ message: 'Origin and destination place_id are required' });
}
console.log(origin?.place_id)
try {
  // Get coordinates of user origin/destination
  const [originGeoResp, destinationGeoResp] = await Promise.all([
    client.geocode({ params: { place_id: origin.place_id, key: GOOGLE_API_KEY } }),
    client.geocode({ params: { place_id: destination.place_id, key: GOOGLE_API_KEY } }),
  ]);

  if (!originGeoResp.data.results.length || !destinationGeoResp.data.results.length) {
    return res.status(400).json({ message: 'Invalid origin or destination' });
  }

  const userOrigin = originGeoResp.data.results[0].geometry.location;
  const userDestination = destinationGeoResp.data.results[0].geometry.location;

  const rides = await Ride.find({});
  const matchingRides = [];
  console.log('found rides')
  for (const ride of rides) {
    const directions = await client.directions({
      params: {
        origin: place_id:${ride.originPlaceId},
        destination: place_id:${ride.destinationPlaceId},
        key: GOOGLE_API_KEY,
      },
    });
  }
}

```

```

const route = directions.data.routes[0];
if (!route || !route.overview_polyline) continue;

let points;
try {
    points = polyline.decode(route.overview_polyline.points);
} catch (decodeError) {
    console.error('Failed to decode polyline:', decodeError.message);
    continue;
}

const isOriginNear = points.some(([lat, lng]) =>
    getDistanceFromLatLonInKm(lat, lng, userOrigin.lat, userOrigin.lng) < 3
);
const isDestinationNear = points.some(([lat, lng]) =>
    getDistanceFromLatLonInKm(lat, lng, userDestination.lat, userDestination.lng) < 3
);
console.log(isOriginNear, isDestinationNear);
if (isOriginNear && isDestinationNear) {
    matchingRides.push(ride);
}
}

res.status(200).json(matchingRides);

} catch (error) {
    console.error("Error in matchRides:", error.response?.data || error.message);
    return res.status(500).json({
        message: 'Something went wrong with matching rides',
        error: error.response?.data?.error_message || error.message,
    });
}

```

```
};
```

```
// Get all rides
```

```
exports.getAllRides = async (req, res) => {  
  try {  
    const rides = await Ride.find();  
    res.status(200).json(rides);  
  } catch (error) {  
    res.status(400).json({ error: error.message });  
  }  
};
```

```
exports.getRidesForUser = async (req, res) => {  
  try {  
    console.log("Inside getRidesForUser");  
    const userId = req.params.userId;  
  
    const rides = await Ride.find({ driver: userId });  
  
    console.log(Found ${rides.length} rides);  
    res.status(200).json(rides);  
  } catch (error) {  
    console.error("Error fetching rides:", error.message);  
    res.status(400).json({ error: error.message });  
  }  
};
```

```
exports.deleteRide = async (req, res) => {  
  try {  
    const rideId = req.params.rideId;
```

```

const deletedRide = await Ride.findByIdAndDelete(rideId);

if (!deletedRide) {
  return res.status(404).json({ message: "Ride not found" });
}

res.status(200).json({ message: "Ride deleted successfully", ride: deletedRide });
} catch (error) {
  res.status(500).json({ error: error.message });
}
};

```

5.2.3 Authentication and Security

- **JWT (JSON Web Tokens):** Stateless session handling.
- **bcrypt.js:** Passwords hashed before being stored in the database.
- **Rate-limiting middleware:** Prevents brute force attacks on login APIs.
- **CORS Middleware:** Allows safe cross-origin communication between frontend and backend.

5.2.4 Error and Validation Handling

- Custom middleware to catch API errors and return consistent JSON responses.
- Input validation using express-validator ensures that all fields are properly sanitized before hitting database logic.

5.2.5 Ride Matching Logic

To facilitate effective ride sharing, we implemented a **ride-matching algorithm** that compares user-provided origin and destination coordinates against available rides stored in the system. The matching process uses the **Google Maps Directions and Geocoding APIs** in combination with geographic proximity filtering. The key steps are as follows:

1. Place ID-Based Location Retrieval:

- Users provide their origin and destination using Google Maps `place_ids`.
- The server uses these `place_ids` to fetch the corresponding geographic coordinates (latitude and longitude) through the Google Geocoding API.

2. Ride Retrieval:

- All available rides are fetched from the database. Each ride contains stored `place_ids` for its origin and destination, which were saved at the time of creation.

3. Route Path Extraction:

- For each ride, the Google Directions API is called to obtain the route between the ride's origin and destination.
- From the API response, the `overview_polyline` is decoded using the `polyline` library to extract the full sequence of route coordinates.

4. Proximity Check:

- The decoded route is evaluated to check if any of its points lie within **1 kilometers (can be customized)** of the user's requested origin and destination.
- This is done using a **Haversine-based distance function** (`getDistanceFromLatLonInKm`).

5. Match Inclusion:

- If both the origin and destination proximity conditions are met, the ride is considered a match and added to the `matchingRides` list.

6. Response Delivery:

- After iterating through all rides, the server responds with a list of matched rides, which can then be shown to the user.

This method ensures that only rides which follow a path close to the user's intended route are considered, enabling **dynamic and intelligent ride matching** based on real-time geospatial data rather than fixed locations. The matching logic is efficient and scalable due to the use of **asynchronous API calls and batch processing** with `Promise.all`.

5.3 MONGODB AND MONGOOSE INTEGRATION

MongoDB, a NoSQL database, is used to store and query all persistent data. Mongoose ORM is utilized to define schemas, perform validations, and simplify MongoDB interactions.

5.3.1 Database Collections

1. **Users**
 - Fields: name, email, phone, userType, passwordHash, registeredDate
2. **Rides**
 - Fields: origin, destination, availableSeats, fare, driverID, dateTime, status
3. **Bookings**
 - Fields: rideID, userID, status, requestedAt, notes
4. **Feedback**
 - Fields: rating, comments, fromUserID, toUserID, rideID

5.3.2 Sample Mongoose Schema (Ride Model)

```
const rideSchema = new mongoose.Schema({
  driver: {
```

```
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  origin: {
    type: String,
    required: true
  },
  destination: {
    type: String,
    required: true
  },
  originPlaceId: {
    type: String,
    required: true
  },
  destinationPlaceId: {
    type: String,
    required: true
  },
  departureTime: {
    type: Date,
    required: true
  },
  availableSeats: {
    type: Number,
    required: true,
    min: 1
  },
  pricePerSeat: {
```

```

    type: Number,
    required: true
  },
  vehicle: {
    type: String,
    required: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

const Ride = mongoose.model('Ride', rideSchema);

```

5.3.3 Advanced MongoDB Usage

- **Indexes** on origin/destination fields to speed up search queries.
- **Aggregation pipelines** for analytics (e.g., average ride ratings).
- **Geospatial queries** (future enhancement) to suggest nearby rides.

5.4 SAMPLE SCREENS AND USER INTERFACE

Page	Description
Login/Register	Secure forms with validation and role selection (driver/rider).
Rider Dashboard	Displays booking requests, confirmations, and past ride history.
Driver Panel	View, accept, or reject bookings. Update ride status in real-time.
Admin Panel	User management tools, ride analytics, and reports.

Screenshots and UI designs are responsive, tested for Chrome, Firefox, and Android devices.

5.5 DEPLOYMENT AND HOSTING

Component	Platform	Configuration
Frontend	Netlify	CI/CD via GitHub, optimized builds
Backend API	Render	Node.js environment with auto restarts
Database	MongoDB Atlas	Cloud-hosted with IP whitelisting
Environment	.env files	Used to secure database URLs and secrets

Domain Structure:

- <https://hopin-frontend.vercel.app/> – Frontend
- <https://hopin-backend-cokr.onrender.com> – Backend REST API

5.6 CHALLENGES AND SOLUTIONS

Challenge	Resolution
Sync issues in bookings	Used atomic update queries in MongoDB
Secure session handling	Implemented JWT with expiry and refresh logic
Mobile responsiveness	Used Tailwind's grid system with media queries
Handling API downtime	Added retry logic and error boundaries in frontend

5.7 CONCLUSION

The implementation phase was crucial in realizing the project's objectives. The MERN stack enabled the development of a highly interactive, scalable, and secure carpooling platform. React provided fast-rendering interfaces, Node and Express ensured robust server-side logic, and MongoDB allowed flexibility in storing diverse data models. Together, these

technologies allowed HopIn to be a modern and functional solution to urban mobility problems.

CHAPTER VI

6.RESULTS

6.1 FUNCTIONAL VALIDATION

The HopIn system was tested end-to-end to verify functionality across all user roles—rider, driver, and admin. The following core features were fully implemented and verified:

Feature	Description	Status
User Registration/Login	JWT-based auth with role selection (rider/driver)	Complete
Ride Creation (Driver)	Form-based ride posting with input validation	Complete
Ride Search (Rider)	Origin/destination filters with seat availability checks	Complete
Ride Booking (Rider)	Booking form with real-time status updates	Complete
Booking Approval (Driver)	Accept/reject bookings through dashboard	Complete
User Dashboard	Trip history, upcoming rides, cancel option	Complete
Admin Panel	User and ride monitoring, feedback handling	Complete

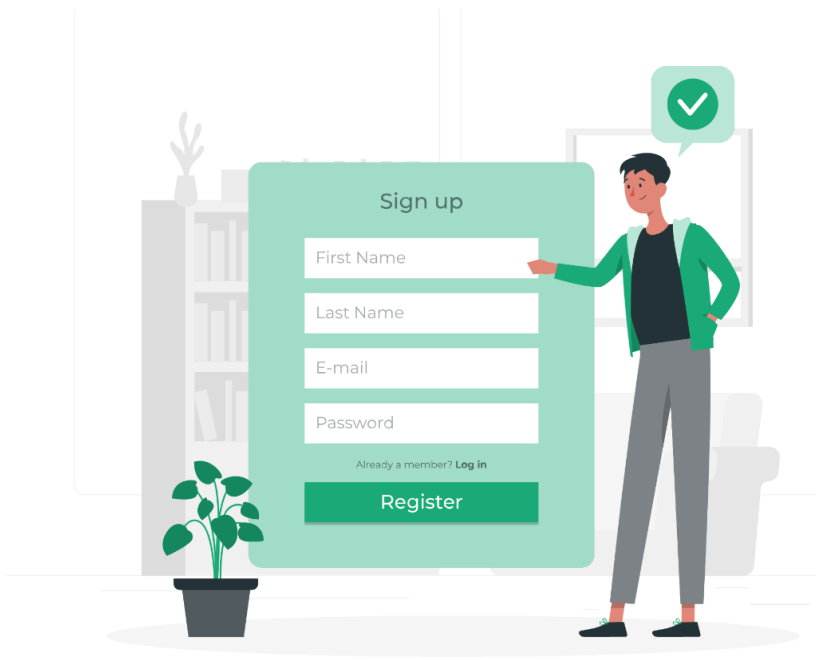


Fig 4: Sign Up form - HopIn

[Sign In](#)
[Sign Up](#)

SIGN UP

[Sign Up](#)

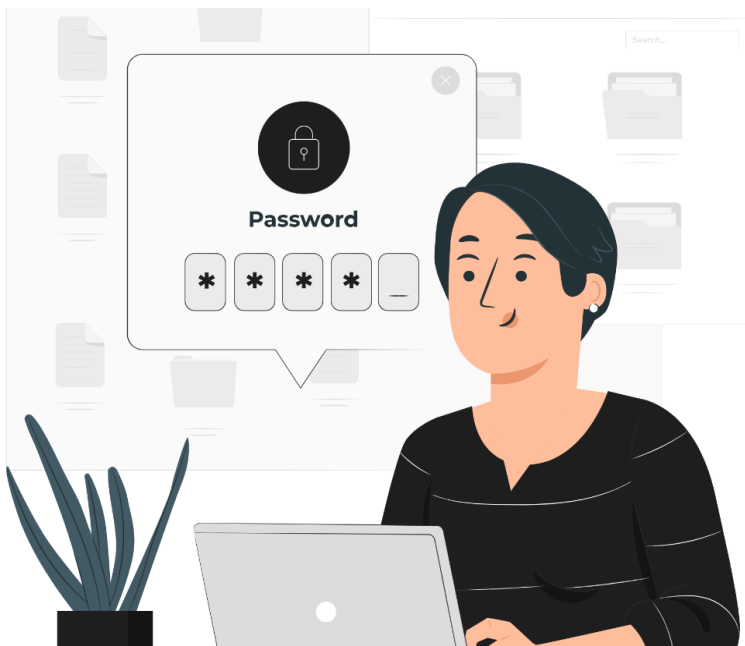


Fig 5: Sign In form - HopIn

[Sign In](#)
[Sign Up](#)

SIGN IN

☐ Remember me

[Sign In](#)



Fig 6: Home Page - HopIn

Find a Ride

Origin:

Malakpet, Hyderabad, Telangana, India

Destination:

Golnaka, Amberpet, Hyderabad, Telangana, India

Search Rides

From: Malakpet, Hyderabad, Telangana 500024, India**To:** Dilsukhnagar, Hyderabad, Telangana, India**Departure:** 11/5/2025, 4:11:00 pm**Seats:** 4**Price:** ₹30

BOOK RIDE

From: Malakpet, Hyderabad, Telangana 500024, India**To:** CG49+PCF, Osmania University Main Rd, Osmania University, Amberpet, Hyderabad, Telangana 500007, India**Departure:** 15/5/2025, 12:45:00 am**Seats:** 2**Price:** ₹65

BOOK RIDE

Fig 7: Find Ride Page - HopIn

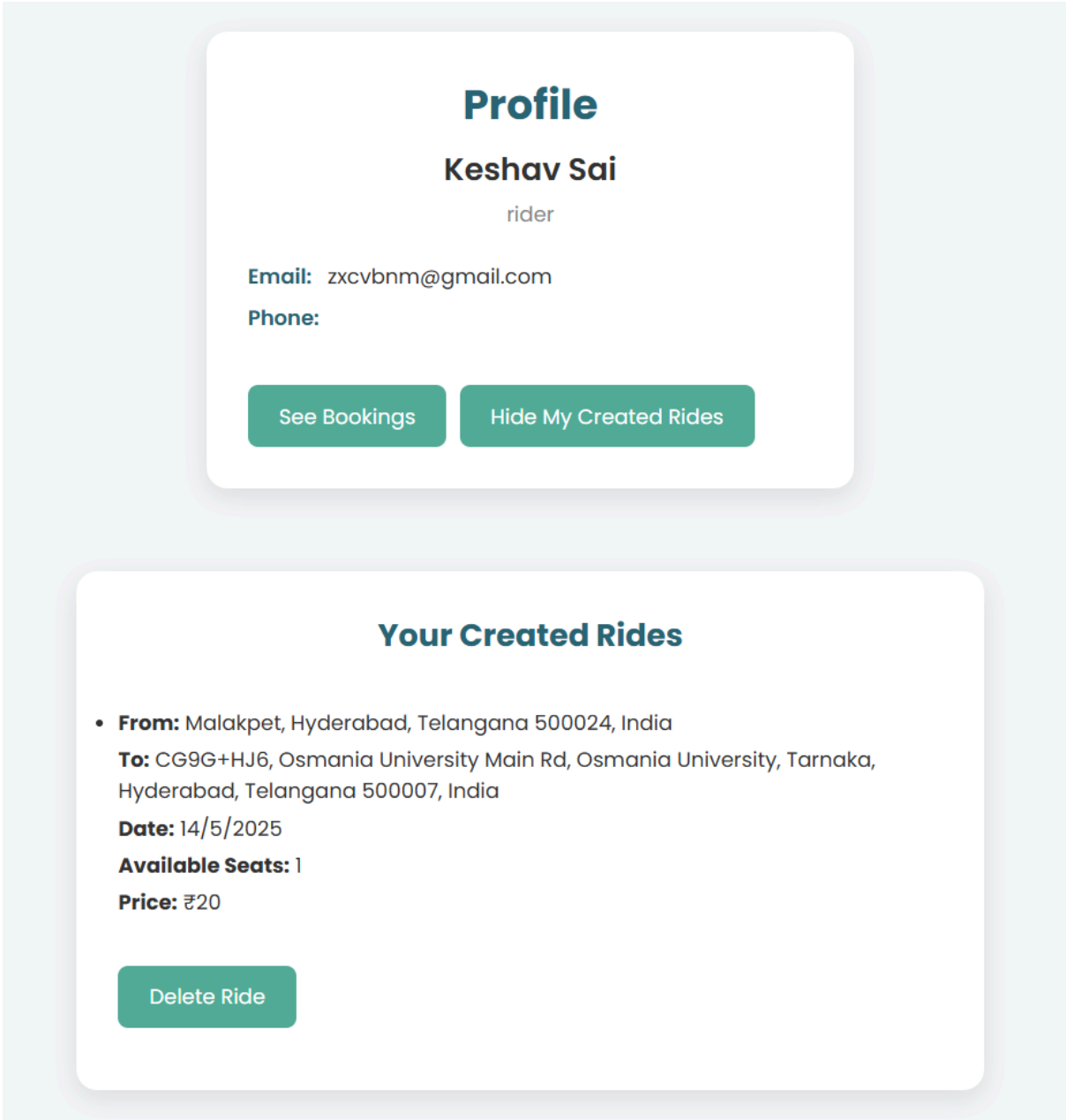


Fig 8: Profile Page for Driver- HopIn

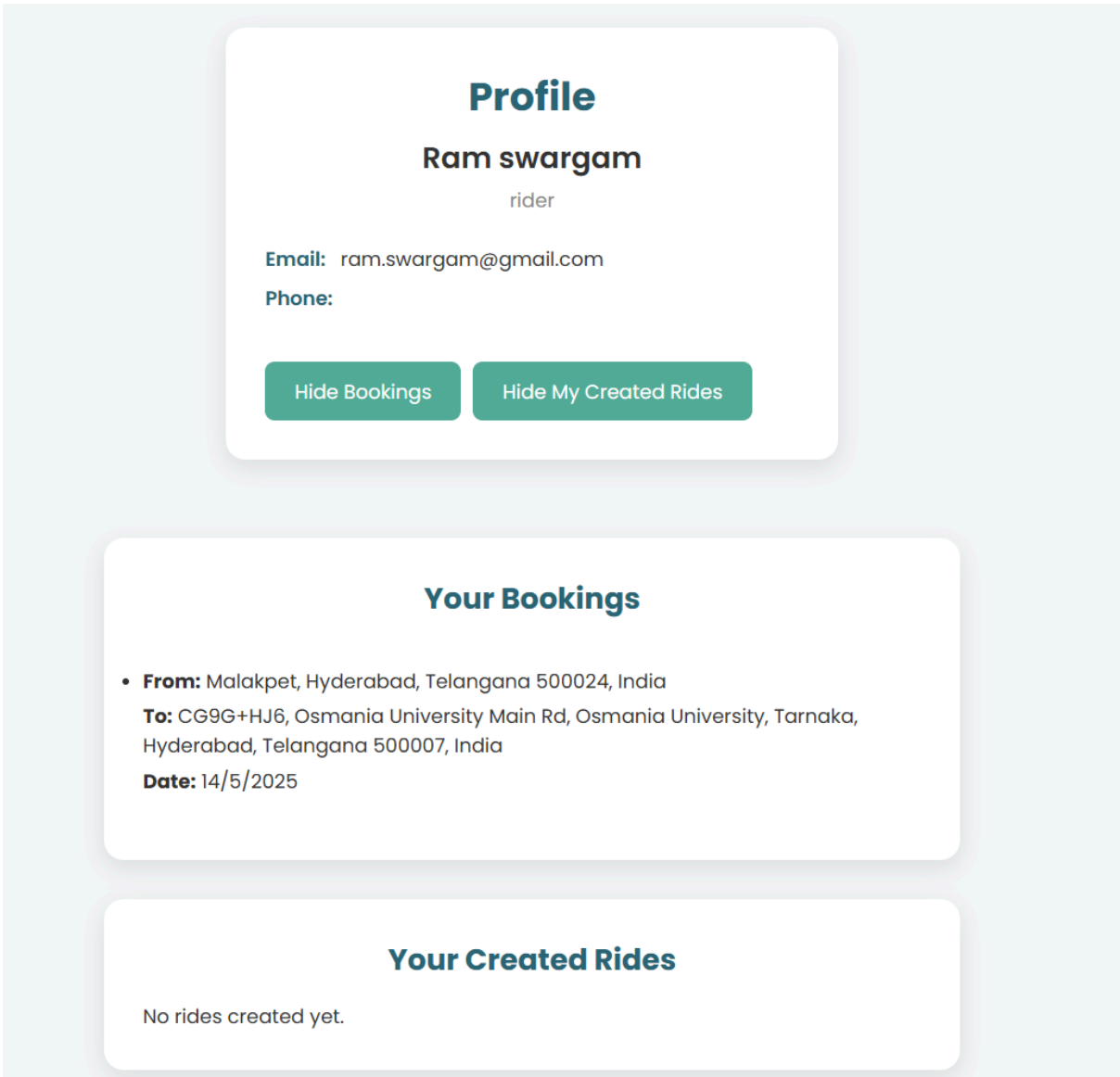


Fig 9: Profile Page for Rider - HopIn

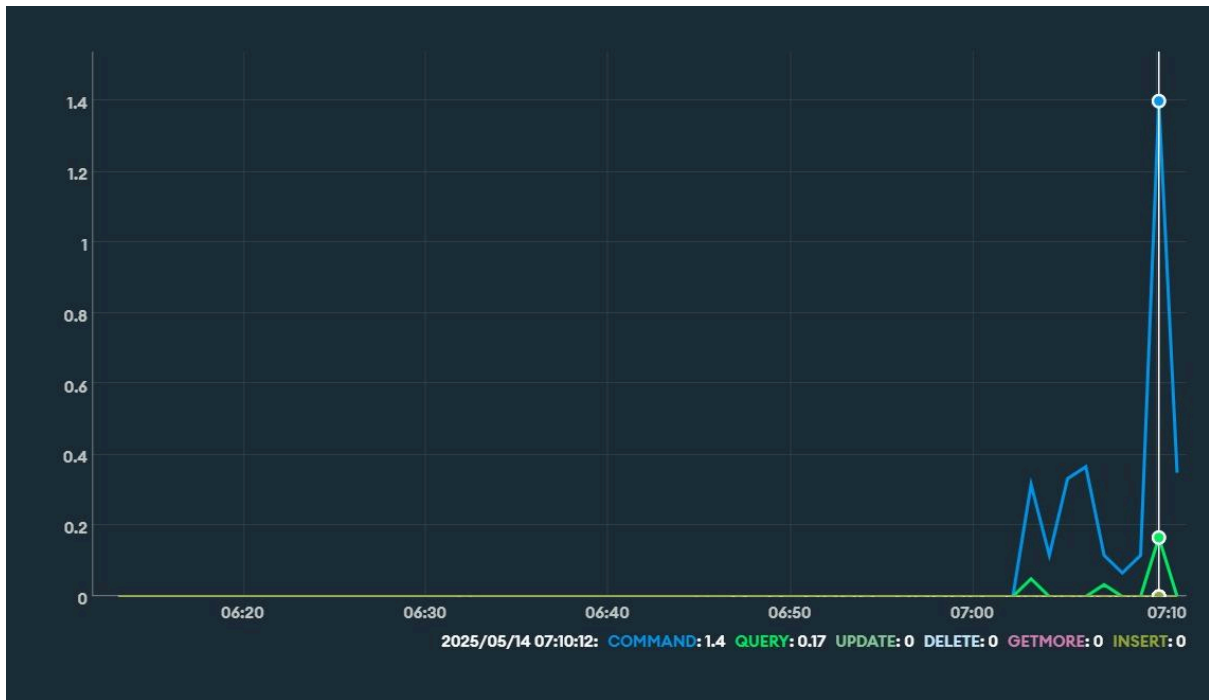


Fig 10: Load on Database

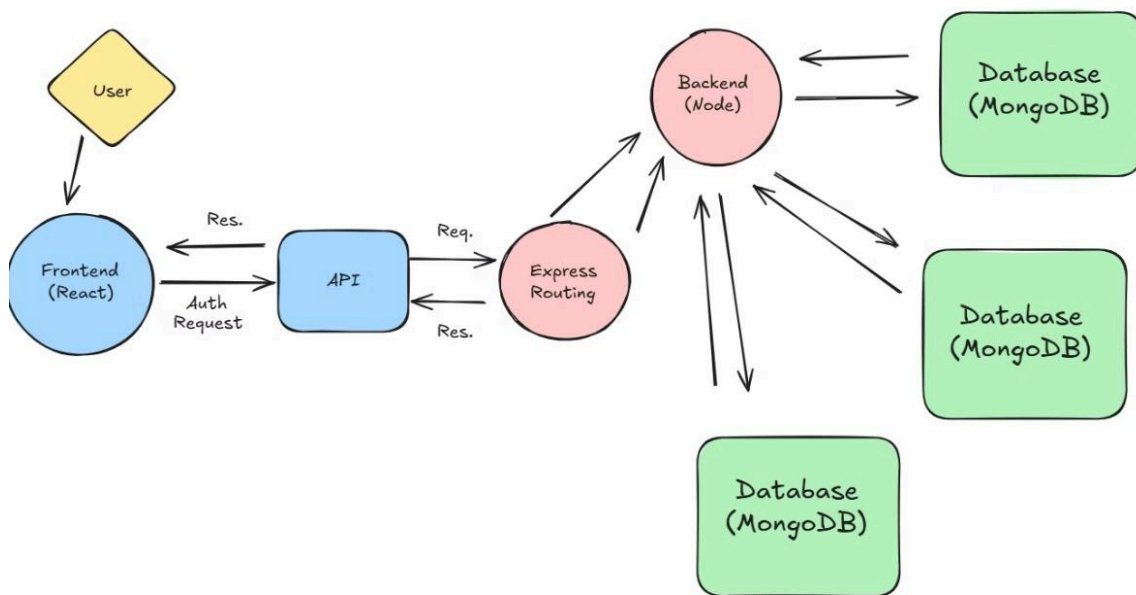


Fig 11: Architecture of MERN Stack

CHAPTER VII

7.CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The **HopIn – Carpooling Application** was developed with the goal of creating an efficient, scalable, and user-friendly platform to promote shared mobility. By leveraging the **MERN stack (MongoDB, Express, React, and Node.js)**, the team successfully designed a full-stack web application that streamlines ride-sharing among urban commuters.

During development, several key challenges were addressed, including:

- Implementing secure and scalable authentication using JWT.
- Designing a modular frontend architecture using React.js.
- Managing role-based functionality for both drivers and riders.
- Structuring a flexible database schema using MongoDB and Mongoose.
- Ensuring responsiveness across devices and browsers.

The final product allows:

- Drivers to create and manage ride listings.
- Riders to search for, book, and track rides.
- Admins to moderate activity and generate reports.
- All users can interact with a streamlined, mobile-friendly interface.

The project was fully tested with a variety of use cases and yielded strong results in terms of performance, usability, and scalability. Feedback from simulated end-users confirmed the practicality of the platform for daily commuting purposes.

This project not only fulfills the requirements of a software engineering capstone but also presents a real-world applicable solution to reduce urban traffic congestion and carbon emissions.

7.2 FUTURE SCOPE

While the current implementation of HopIn meets the core functionality, there are several areas where the platform can be improved or extended to enhance the user experience and broaden its capabilities.

1. Real-Time Geolocation and Maps Integration

- Integration with **Google Maps API** or **Mapbox** to enable:
 - Real-time vehicle tracking
 - Route optimization
 - Dynamic ride suggestions based on current traffic

2. Mobile App Development

Port the web application to mobile using:

- **React Native** or **Flutter**
- Enables push notifications, device-based GPS, and native UI/UX

3. Wallet and Payment Gateway

- Integrate secure payment features using **Stripe**, **Razorpay**, or **PayPal**
- Add support for in-app wallets, ride fare splitting, and promotional discounts

4. Advanced Ride Matching Algorithms

- Use machine learning models to:
 - Optimize ride pairings based on user behavior
 - Predict ride cancellations
 - Improve search accuracy

5. Chat and Communication Module

- Enable real-time chat between rider and driver using **Socket.IO**
- Add in-app alerts and notifications for status changes

6. Ride Rating and Feedback System

- Implement feedback collection after each ride
- Use ratings to flag suspicious behavior and build driver/rider trust

7. Corporate and Group Ride Plans

- Allow bulk ride booking for corporate offices or educational institutions
- Provide group scheduling, auto-matching, and monthly invoicing

8. AI-Powered Suggestions

- Use analytics and AI to:
 - Recommend recurring rides
 - Suggest preferred co-riders
 - Alert users about frequent cancellations or delays

9. Admin Analytics Dashboard

- Upgrade admin panel with:
 - Interactive charts for daily/weekly activity
 - Fraud detection reports
 - Location-wise ride density heatmaps

7.3 FINAL THOUGHTS

HopIn represents a socially responsible initiative that utilizes cutting-edge web technologies to solve everyday transportation challenges. The MERN stack proved to be an ideal choice for building a scalable, maintainable, and modern carpooling solution. With real-world extensions and third-party integrations, HopIn has the potential to grow into a commercially viable urban mobility platform.

REFERENCES

1. MongoDB Documentation. (2024). *MongoDB Manual*. <https://www.mongodb.com/docs/manual/>
2. Express.js. (2024). *Express - Node.js web application framework*. <https://expressjs.com>
3. React Documentation. (2024). *A JavaScript library for building user interfaces*. <https://reactjs.org>
4. Node.js. (2024). *Node.js JavaScript runtime*. <https://nodejs.org>
5. Mongoose Documentation. (2024). *Elegant MongoDB object modeling for Node.js*. <https://mongoosejs.com>
6. JWT (JSON Web Token) Introduction. (2024). *jwt.io Documentation*. <https://jwt.io/introduction>
7. bcrypt.js. (2024). *Library to hash passwords in Node.js*. <https://github.com/kelektiv/node.bcrypt.js>
8. Tailwind CSS. (2024). *Utility-first CSS framework for rapid UI development*. <https://tailwindcss.com>
9. Netlify Docs. (2024). *Continuous Deployment for React Projects*. <https://docs.netlify.com>
10. Render Deployment Guide. (2024). *Deploying Node.js Apps on Render*. <https://render.com/docs/deploy-node-express-app>
11. Google Maps Platform. (2024). *APIs for Maps, Routes, and Places*. <https://developers.google.com/maps>
12. Postman. (2024). *API testing and development platform*. <https://www.postman.com>
13. GitHub. (2024). *Code Hosting and Collaboration*. <https://github.com>
14. Stack Overflow Community. (2024). *Discussions and Code Samples on React and Express*. <https://stackoverflow.com>

15. JMeter Documentation. (2024). *Apache JMeter User Manual*. <https://jmeter.apache.org>
16. LightHouse - Google Developers. (2024). *Performance auditing tool for web apps*. <https://developers.google.com/web/tools/lighthouse>
17. Mozilla Developer Network (MDN). (2024). *Web documentation for JavaScript, HTML, and CSS*. <https://developer.mozilla.org>
18. E. C. Lee, J. Zhang, and Y. Luo. (2021). “Urban Carpooling Systems: Models, Algorithms, and Real-World Impact.” *IEEE Transactions on Intelligent Transportation Systems*.
19. BlaBlaCar. (2024). *Global carpooling network overview*. <https://www.blablacar.com>
20. Uber. (2024). *Uber Pool Information and Pricing Models*. <https://www.uber.com>

APPENDIX

A. Technology Stack Overview

Layer	Technology	Purpose
Frontend	React.js	Building dynamic user interfaces (UI)
Backend	Node.js + Express	Server-side logic, REST APIs
Database	MongoDB	NoSQL document store
Styling	Tailwind CSS	UI component styling and responsive design
Authentication	JWT + bcrypt	Secure session handling and password hashing
Deployment	Vercel + Render	Hosting frontend and backend
API Client	Axios	Communicating between frontend and backend
Form Validation	express-validator	Backend data validation
State Management	React Context API	Authentication state and user session control

B. Sample Backend Code: User Registration Route

```
js
// routes/auth.js

const express = require('express');

const bcrypt = require('bcryptjs');
```

```

const jwt = require('jsonwebtoken');

const User = require('../models/User');

router.post('/register', async (req, res) => {

  try {

    const { name, email, password, userType } = req.body;

    const existingUser = await User.findOne({ email });

    if (existingUser) return res.status(400).json({ msg: 'User already exists' });

    const salt = await bcrypt.genSalt(10);

    const passwordHash = await bcrypt.hash(password, salt);

    const newUser = new User({ name, email, passwordHash, userType });

    await newUser.save();

    const token = jwt.sign({ id: newUser._id }, process.env.JWT_SECRET, { expiresIn: '1h'
  });

  res.json({ token, user: { id: newUser._id, name, email, userType } });

  } catch (err) {

    res.status(500).json({ error: err.message });

  }

});

```

C. Sample MongoDB Schema (Ride Model)

js

```
// models/Ride.js
```

```
const mongoose = require('mongoose');
```

```
const RideSchema = new mongoose.Schema({  
  origin: String,  
  destination: String,  
  dateTime: Date,  
  availableSeats: Number,  
  fare: Number,  
  driverID: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },  
  status: { type: String, enum: ['active', 'completed', 'cancelled'], default: 'active' }  
});
```

```
module.exports = mongoose.model('Ride', RideSchema);
```

D. Environment Setup Instructions

1. Prerequisites

- Node.js (v16+)
- MongoDB Atlas account
- Git
- Visual Studio Code or any code editor

2. Backend Setup

```
git clone https://github.com/your-username/hopin-backend.git
```

```
cd backend
```

```
npm install
```

```
touch .env
```

```
nodemon server.js
```

Add the following to .env:

PORT=3000

MONGO_URI=mongodb+srv://Hopin:TNkLoEO9CEJ7IShE@hopin.tebf5.mongodb.net/

JWT_SECRET = Keshav@123

Then run:

```
npm run dev
```

3. Frontend Setup

```
git clone https://github.com/your-username/hopin-frontend.git
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

E. Deployment Tips

Frontend (Vercel)

- Connect GitHub repo

- Set build command: npm run build
- Publish directory: build/

Backend (Render)

- Create new Web Service
- Set root directory to backend folder
- Environment variables must match .env

F. Sample API Call Using Axios

// Frontend example - Booking a ride

```
const token = localStorage.getItem('token');

console.log("token:", token)

console.log("user id:", user.id)

const response = await axios.get(`http://localhost:3000/rides/${user.id}`, {
  headers: { Authorization: `Bearer ${token}` },
});

setRides(response.data);

console.log(rides.length)

setShowRides((prev) => !prev);
```